

Implementazione e analisi sperimentale dell'algoritmo di Brandes per la betweenness centrality

Sebastiano Becchetti

25 giugno 2026

Indice

1	Introduzione al problema e soluzioni precedenti	2
1.1	La betweenness centrality	2
1.2	L'approccio classico e il suo costo	2
1.3	Implementazione e limiti dell'approccio standard	3
1.4	Il contributo di Brandes	5
1.5	Obiettivo	5
2	Implementazione proposta	6
2.1	La relazione ricorsiva sulle dipendenze	6
2.2	Pseudocodice	7
2.3	Esempio passo-passo: il diamante a 5 vertici	7
2.4	Generazione dei grafi di test	9
2.5	Il calcolo dei cammini minimi	10
2.6	L'accumulo all'indietro	10
2.7	Ottimizzazioni a costante	11
3	Estensione ai grafi pesati: l'algoritmo di Dijkstra	12
3.1	Inadeguatezza della BFS	12
3.2	Le modifiche al codice	12
3.3	La cancellazione pigra	13
3.4	L'accumulo all'indietro nel caso pesato	13
3.5	Complessità	14
3.6	Verifica ed esempio visivo	15
4	Risultati sperimentali	16
4.1	Impostazione degli esperimenti	16
4.2	Verifica di correttezza	16
4.3	Interpretazione visiva della betweenness	17
4.4	Analisi della complessità	17
4.4.1	Il sovraccarico lineare per sorgente	18
4.5	Riproduzione dell'esperimento di Brandes	18
5	Conclusioni	19
5.1	Sintesi del lavoro	19

Capitolo 1

Introduzione al problema e soluzioni precedenti

1.1 La betweenness centrality

Nell'analisi delle reti reali — sociali, biologiche, infrastrutturali — ricorre l'esigenza di quantificare la rilevanza di un singolo nodo per la costruzione del grafo, e quindi quanto sia importante per la costruzione dei cammini minimi tra i vari nodi. Tra le misure proposte a tale scopo, una delle più diffuse è la *betweenness centrality*, formalizzata da Freeman [2] a partire da idee precedentemente introdotte da Anthonisse [3].

Il principio su cui si fonda la misura è il seguente: la centralità di un nodo è tanto maggiore quanto più frequentemente esso è presente sui cammini minimi che collegano le altre coppie di nodi. Un nodo con *betweenness* elevata costituisce un punto di passaggio pressoché obbligato, la cui rimozione o congestione compromette la comunicazione tra ampie porzioni della rete.

Formalmente, dato un grafo $G = (V, E)$ con $n = |V|$ vertici e $m = |E|$ archi, si definisce la *betweenness centrality* di un vertice v come

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}, \quad (1.1)$$

dove σ_{st} è il numero totale di cammini minimi tra s e t , mentre $\sigma_{st}(v)$ è il numero di quei cammini che passano per v . Il rapporto $\sigma_{st}(v)/\sigma_{st}$ esprime la frazione di cammini minimi tra s e t che dipendono da v : in presenza di più cammini minimi alternativi, il contributo di v viene ripartito di conseguenza.

1.2 L'approccio classico e il suo costo

Il calcolo diretto della formula (1.1) richiede, per ogni coppia di vertici (s, t) , di conoscere il numero e la composizione dei cammini minimi. Gli approcci classici procedono in due fasi:

1. calcolo di tutte le distanze e di tutti i cammini minimi tra tutte le coppie (*all-pairs shortest paths*);
2. somma esplicita, per ogni coppia (s, t) e per ogni vertice intermedio v , del contributo $\sigma_{st}(v)/\sigma_{st}$.

La seconda fase costituisce il principale fattore limitante. La somma esplicita richiede, per ciascuno degli n vertici v , di scorrere tutte le $\Theta(n^2)$ coppie ordinate (s, t) : il corpo interno viene quindi eseguito $\Theta(n^3)$ volte, da cui il costo temporale $\Theta(n^3)$. A ciò si aggiunge il costo

in memoria: per valutare ogni rapporto $\sigma_{st}(v)/\sigma_{st}$ occorre aver memorizzato i conteggi σ_{st} per tutte le coppie di vertici, ovvero uno spazio $O(n^2)$. Per reti dell'ordine di decine di migliaia di nodi è proprio l'occupazione di memoria quadratica a divenire proibitiva, ancor prima del costo temporale. La sezione seguente ne presenta l'implementazione, il relativo pseudocodice e i tempi misurati.

1.3 Implementazione e limiti dell'approccio standard

Per misurarne concretamente i limiti, l'approccio classico è stato implementato in C++ (file `src/standard.cpp`), seguendo fedelmente le due fasi:

- **Fase 1 (all-pairs)**: una visita in ampiezza da ciascuna sorgente calcola e *memorizza* le matrici complete delle distanze $d[s][t]$ e dei conteggi di cammini minimi $\sigma[s][t]$ per tutte le coppie;
- **Fase 2 (somma esplicita)**: per ogni vertice intermedio v e per ogni coppia (s, t) , il *criterio di Bellman* stabilisce se v giace su un cammino minimo da s a t , nel qual caso si accumula la relativa pair-dependency.

Algorithm 1 Approccio standard: all-pairs seguito da somma esplicita.

```

1: Fase 1: cammini minimi tra tutte le coppie
2: for  $s \in V$  do ▷ una visita in ampiezza per sorgente
3:    $d[s][t] \leftarrow -1, \sigma[s][t] \leftarrow 0$  per ogni  $t$ 
4:    $d[s][s] \leftarrow 0, \sigma[s][s] \leftarrow 1$ 
5:   visita in ampiezza da  $s$  che aggiorna  $d[s][\cdot]$  e  $\sigma[s][\cdot]$ 
6: end for
7: Fase 2: somma esplicita delle pair-dependency
8:  $C_B[v] \leftarrow 0$  per ogni  $v \in V$ 
9: for  $v \in V$  do
10:  for  $s \in V, s \neq v$  do
11:    for  $t \in V, t \neq s, t \neq v$  do
12:      if  $d[s][v] + d[v][t] = d[s][t]$  then ▷ criterio di Bellman
13:         $C_B[v] += \frac{\sigma[s][v] \cdot \sigma[v][t]}{\sigma[s][t]}$ 
14:      end if
15:    end for
16:  end for
17: end for
18: per grafi non orientati:  $C_B[v] \leftarrow C_B[v]/2$  per ogni  $v$ 

```

Si ritrovano i due costi: la Fase 2 è $\Theta(n^3)$ (tre cicli su V), mentre le matrici d e σ della Fase 1 impongono uno spazio $\Theta(n^2)$. Il vero collo di bottiglia è quest'ultimo: già per reti di poche decine di migliaia di nodi le due matrici non entrano in memoria, rendendo l'approccio inapplicabile indipendentemente dal tempo.

La Figura 1.1 riporta i tempi misurati dall'implementazione, con lo stesso schema sperimentale usato nel seguito (una curva per densità, sweep su n). Il range di n è qui limitato a 1000: il costo $\Theta(n^3)$ rende già proibitivo spingersi oltre.

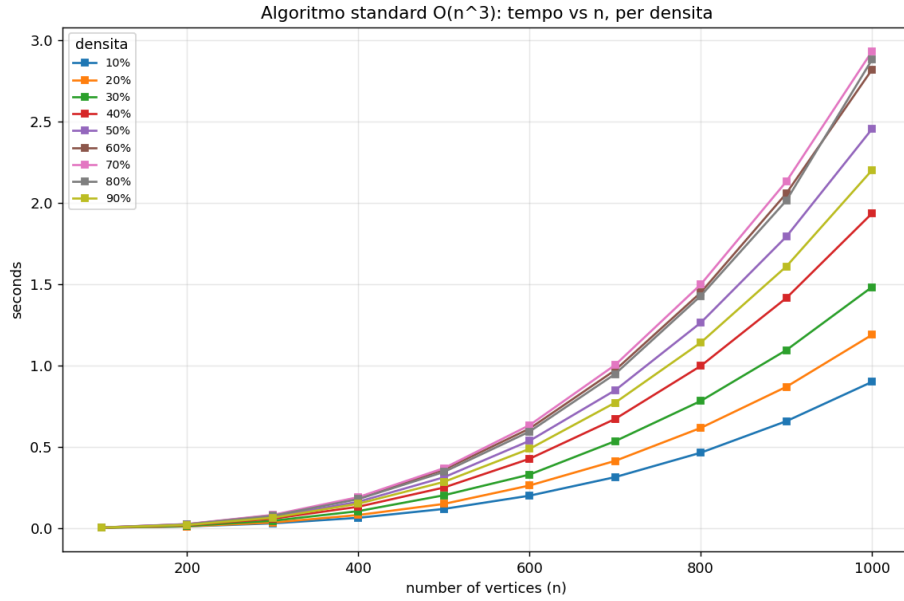


Figura 1.1: Tempo di esecuzione dell'approccio standard $O(n^3)$ in funzione del numero di vertici, una curva per ciascuna densità.

Il confronto diretto con l'algoritmo di Brandes (Figura 1.2) chiarisce dove stia il vantaggio. Sui grafi qui generati, a densità 10%–90%, si ha $m = \Theta(n^2)$: in questo regime denso anche Brandes costa $O(nm) = O(n^3)$, e i tempi sono dunque dello stesso ordine (lo standard resta più lento per la costante della somma esplicita). Il guadagno decisivo di Brandes emerge invece sui grafi *sparsi* — tipici delle reti reali — e, soprattutto, nello spazio: $O(n + m)$ lineare contro $O(n^2)$.

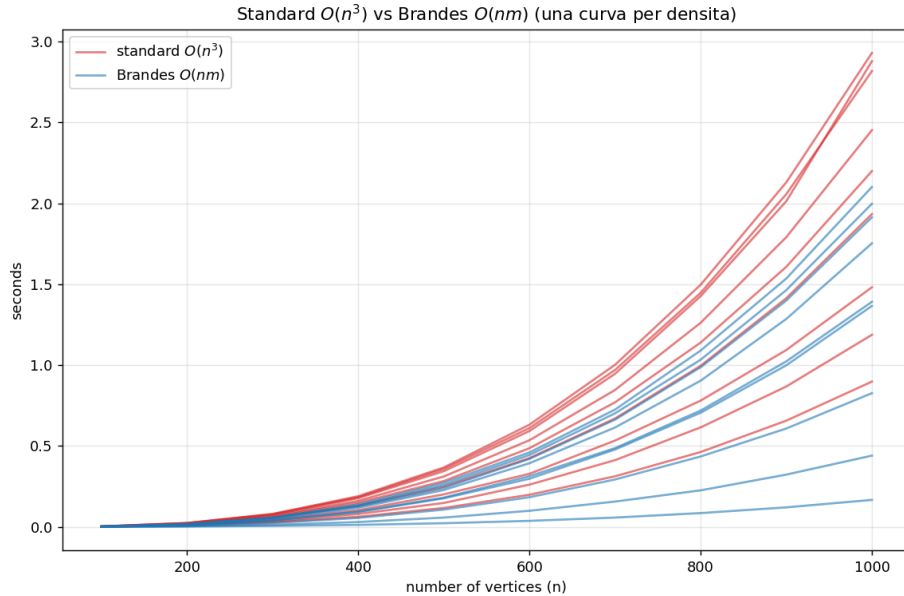


Figura 1.2: Confronto dei tempi: approccio standard $O(n^3)$ (rosso) contro algoritmo di Brandes $O(nm)$ (blu), una curva per densità sul range di n comune.

1.4 Il contributo di Brandes

Nel 2001 Brandes [1] propone un algoritmo che riduce sensibilmente il costo computazionale senza introdurre alcuna approssimazione. L'idea centrale consiste nel non sommare le dipendenze coppia per coppia, bensì nell'*accumularle* in forma aggregata considerando una sola sorgente per volta, attraverso una relazione ricorsiva sulle dipendenze discussa nel Capitolo 2.

I costi ottenuti sono:

- **grafi non pesati:** $O(nm)$ tempo, tramite una BFS (*Breadth-First Search*) da ciascuna sorgente;
- **grafi pesati:** $O(nm + n^2 \log n)$ tempo, usando l'algoritmo di Dijkstra con *Fibonacci heap* [4];
- **spazio:** $O(n + m)$ in entrambi i casi, ovvero lineare.

La riduzione dello spazio da $O(n^2)$ a $O(n + m)$ è l'elemento che rende l'algoritmo applicabile a reti di grande dimensione.

1.5 Obiettivo

La presente tesina documenta un'implementazione in C++ dell'algoritmo di Brandes per grafi non orientati, ne verifica la correttezza mediante confronto con la libreria NetworkX [5] e ne analizza sperimentalmente la complessità, riproducendo lo schema di esperimento presentato nell'articolo originale. La trattazione muove dal caso *non pesato* (Capitolo 2), in cui i cammini minimi sono determinati tramite la BFS, e si estende successivamente al caso *pesato* (Capitolo 3), in cui la costruzione dei cammini minimi è sostituita dall'algoritmo di Dijkstra; di quest'ultima variante si illustrano il funzionamento, le modifiche apportate al codice e l'andamento sperimentale dei tempi di esecuzione al variare della densità. Vengono inoltre discusse alcune ottimizzazioni a costante (riutilizzo dei buffer di memoria) che, pur non modificando la complessità asintotica $O(nm)$, riducono sensibilmente i tempi di esecuzione.

Capitolo 2

Implementazione proposta

Il presente capitolo descrive in dettaglio l'algoritmo di Brandes per grafi non pesati, ponendo ciascun passo dello pseudocodice in corrispondenza con la relativa porzione del codice C++ contenuto nel file `src/brandes.cpp`.

2.1 La relazione ricorsiva sulle dipendenze

Il fulcro dell'algoritmo è la nozione di *dipendenza* di una sorgente s da un vertice v :

$$\delta_s(v) = \sum_{t \in V} \frac{\sigma_{st}(v)}{\sigma_{st}}. \quad (2.1)$$

Sommando $\delta_s(v)$ su tutte le sorgenti si riottiene $C_B(v) = \sum_s \delta_s(v)$. Brandes dimostra che la dipendenza soddisfa la relazione ricorsiva

$$\delta_s(v) = \sum_{w: v \in P_s(w)} \frac{\sigma_{sv}}{\sigma_{sw}} (1 + \delta_s(w)), \quad (2.2)$$

dove $P_s(w)$ è l'insieme dei *predecessori* di w lungo i cammini minimi da s , cioè i vertici v adiacenti a w tali che $d(s, v) + 1 = d(s, w)$. La relazione (2.2) permette di calcolare tutte le dipendenze di una sorgente con una sola passata all'indietro sui vertici, in ordine di distanza non crescente.

2.2 Pseudocodice

Algorithm 2 Betweenness centrality (Brandes, grafi non pesati)

```

1:  $C_B[v] \leftarrow 0$  per ogni  $v \in V$ 
2: for  $s \in V$  do
3:   inizializza pila  $S \leftarrow \emptyset$ , liste  $P[w] \leftarrow \emptyset$ 
4:    $\sigma[t] \leftarrow 0$ ,  $\sigma[s] \leftarrow 1$ 
5:    $d[t] \leftarrow -1$ ,  $d[s] \leftarrow 0$ 
6:   coda  $Q \leftarrow \{s\}$ 
7:   while  $Q \neq \emptyset$  do ▷ BFS
8:     estrai  $v$  da  $Q$ ; inserisci  $v$  in  $S$ 
9:     for ogni vicino  $w$  di  $v$  do
10:      if  $d[w] < 0$  then ▷  $w$  trovato per la prima volta
11:         $d[w] \leftarrow d[v] + 1$ ; inserisci  $w$  in  $Q$ 
12:      end if
13:      if  $d[w] = d[v] + 1$  then ▷ cammino minimo verso  $w$  via  $v$ 
14:         $\sigma[w] \leftarrow \sigma[w] + \sigma[v]$ 
15:        inserisci  $v$  in  $P[w]$ 
16:      end if
17:    end for
18:  end while
19:   $\delta[v] \leftarrow 0$  per ogni  $v$ 
20:  while  $S \neq \emptyset$  do ▷ accumulo all'indietro
21:    estrai  $w$  dalla cima di  $S$ 
22:    for ogni  $v \in P[w]$  do
23:       $\delta[v] \leftarrow \delta[v] + \frac{\sigma[v]}{\sigma[w]}(1 + \delta[w])$ 
24:    end for
25:    if  $w \neq s$  then  $C_B[w] \leftarrow C_B[w] + \delta[w]$ 
26:    end if
27:  end while
28: end for
29: per grafi non orientati:  $C_B[v] \leftarrow C_B[v]/2$  per ogni  $v$ 

```

2.3 Esempio passo-passo: il diamante a 5 vertici

Per rendere concreto lo pseudocodice si segue l'esecuzione dell'algoritmo su un piccolo grafo non pesato, mostrando l'evoluzione di tutti i vettori coinvolti. Si considera il grafo di Figura 2.1, con $n = 5$ vertici e gli archi

$$\{0, 1\}, \{0, 2\}, \{1, 3\}, \{2, 3\}, \{3, 4\}.$$

La struttura a diamante fra 0, 1, 2, 3 è scelta deliberatamente: da 0 a 3 esistono *due* cammini minimi, 0–1–3 e 0–2–3, così da osservare il conteggio σ assumere un valore maggiore di 1.

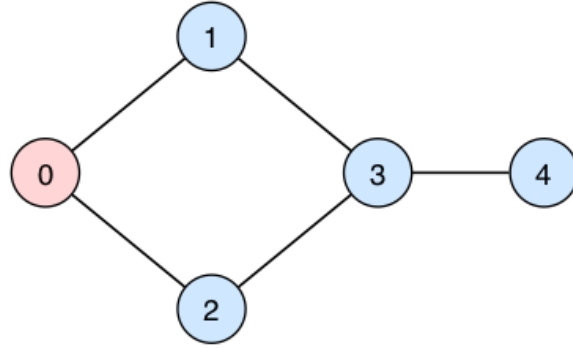


Figura 2.1: Il diamante a 5 vertici. In rosso la sorgente $s = 0$ da cui si avvia la visita.

Fase 1: il calcolo dei cammini minimi dalla sorgente $s = 0$

All'avvio $d[0] = 0$, $\sigma[0] = 1$, mentre per ogni altro vertice $d = -1$ (non visitato) e $\sigma = 0$; la coda contiene $\{0\}$. Le due tabelle seguenti riportano lo stato *successivo* all'estrazione di ciascun nodo dalla coda (il valore -1 indica un vertice non ancora raggiunto).

Tabella 2.1: Evoluzione di **dist** (calcolo dei cammini minimi da $s = 0$).

nodo estratto	d_0	d_1	d_2	d_3	d_4
– init –	0	-1	-1	-1	-1
0	0	1	1	-1	-1
1	0	1	1	2	-1
2	0	1	1	2	-1
3	0	1	1	2	3
4	0	1	1	2	3

Tabella 2.2: Evoluzione di **sigma** (numero di cammini minimi).

nodo estratto	σ_0	σ_1	σ_2	σ_3	σ_4
– init –	1	0	0	0	0
0	1	1	1	0	0
1	1	1	1	1	0
2	1	1	1	2	0
3	1	1	1	2	2
4	1	1	1	2	2

Il passaggio cruciale è l'estrazione del nodo 2: poiché $d_3 = d_2 + 1$, il nodo 3 riceve un *secondo* contributo e σ_3 passa da 1 a 2, in corrispondenza dei due cammini minimi del diamante. Tale valore 2 si propaga poi al nodo 4. Al termine della visita la pila **stack** contiene (dal fondo alla cima)

$$\text{stack} = [0, 1, 2, 3, 4].$$

Fase 2: accumulo delle dipendenze

Si svuota la pila **stack** estraendo i nodi dalla cima (da 4 a 0). Per ogni nodo w si aggiornano i suoi predecessori v , quelli con $d_v = d_w - 1$, mediante

$$\delta_v += \frac{\sigma_v}{\sigma_w} (1 + \delta_w).$$

Tabella 2.3: Evoluzione di **delta** durante l'accumulo all'indietro ($s = 0$).

w corrente	δ_0	δ_1	δ_2	δ_3	δ_4
(start)	0	0	0	0	0
4	0	0	0	1	0
3	0	1	1	1	0
2	2	1	1	1	0
1	4	1	1	1	0
0	4	1	1	1	0

Dettaglio dei passaggi più significativi:

- $w = 4$: unico predecessore 3; $\delta_3 += \frac{\sigma_3}{\sigma_4}(1 + \delta_4) = \frac{2}{2}(1 + 0) = 1$.
- $w = 3$: due predecessori, 1 e 2; ciascuno riceve $\frac{\sigma_v}{\sigma_3}(1 + \delta_3) = \frac{1}{2}(1 + 1) = 1$. Qui la dipendenza si ripartisce fra i due cammini minimi alternativi.
- $w = 2$ e $w = 1$: il predecessore è la sorgente 0; ognuno aggiunge $\frac{1}{1}(1 + 1) = 2$, portando δ_0 a 4.

Poiché δ_0 riguarda la sorgente stessa non viene sommato a C_B . Il contributo della sola sorgente $s = 0$ è quindi

$$C_B^{(s=0)} = [0, 1, 1, 1, 0].$$

Risultato finale

Ripetendo la procedura per tutte le sorgenti $s = 0, \dots, 4$, sommando i contributi e dividendo per 2 (grafo non orientato) si ottiene

$$C_B = [0.5, 1.0, 1.0, 3.5, 0.0].$$

Il nodo 3 presenta la betweenness nettamente maggiore: è il *ponte* del grafo, attraversato dalla quasi totalità dei cammini minimi. I nodi 1 e 2, simmetrici, hanno valore identico, mentre il nodo 4, foglia, ha betweenness nulla.

2.4 Generazione dei grafi di test

Le prove sperimentali richiedono la generazione di grafi casuali *connessi*: un grafo disconnesso falserebbe l'analisi dei tempi, in quanto la visita in ampiezza da una data sorgente esplorerebbe la sola componente cui essa appartiene. La funzione **genera_grafo** costruisce dapprima uno *spanning tree* casuale, che garantisce la connessione mediante $n - 1$ archi, e aggiunge successivamente archi casuali fino a raggiungere il numero m richiesto, escludendo cappi e archi multipli.

Listing 2.1: Generazione di un grafo connesso (estratto).

```

1 // 1) spanning tree casuale: ogni nodo i si collega a un nodo
  precedente
2 for (int i = 1; i < n; i++) {
3     std::uniform_int_distribution<int> prec(0, i - 1);
4     aggiungi(i, prec(gen));
5 }
6 // 2) archi restanti scelti a caso fino a m archi totali
7 std::uniform_int_distribution<int> dist(0, n - 1);
8 while ((int)g.U.size() < m) {
9     aggiungi(dist(gen), dist(gen));
10 }

```

La lambda `aggiungi` normalizza l'arco (u, v) con $u < v$, scarta i duplicati tramite un `std::set` e aggiorna le liste di adiacenza.

2.5 Il calcolo dei cammini minimi

Il blocco che realizza il calcolo dei cammini minimi determina, per ciascun vertice, la distanza `dist` e il numero di cammini minimi `sigma`, e inserisce i vertici nell'ordine di visita nella pila `stack` (la pila S dello pseudocodice). La coda dei nodi ancora da espandere è chiamata `queue`. Si osserva la piena corrispondenza con le righe 7–14 dello pseudocodice.

Per la pila S si è scelto `std::stack`, che corrisponde direttamente alla struttura dello pseudocodice. Una soluzione equivalente sarebbe un `std::vector` riempito durante la visita e percorso all'indietro: le due realizzazioni hanno la stessa complessità e, in pratica, prestazioni indistinguibili; si è preferito `std::stack` per la maggiore aderenza all'algoritmo descritto.

Listing 2.2: Calcolo dei cammini minimi con il relativo conteggio (BFS).

```

1 while (!queue.empty()) {
2     int nodo = queue.front();
3     queue.pop();
4     stack.push(nodo); // inserimento nella pila
5     for (int vicino : g.adj[nodo]) {
6         if (dist[vicino] == -1) { // primo ritrovamento
7             dist[vicino] = dist[nodo] + 1;
8             queue.push(vicino);
9         }
10        if (dist[vicino] == dist[nodo] + 1) // cammino minimo verso
            vicino
11            sigma[vicino] += sigma[nodo];
12    }
13 }

```

Il vettore `sigma` è dichiarato di tipo `long long`: su grafi di grandi dimensioni ed elevata densità il numero di cammini minimi può eccedere l'intervallo rappresentabile da `int`, e tale scelta previene l'overflow.

2.6 L'accumulo all'indietro

Terminato il calcolo dei cammini minimi, i vertici vengono esaminati in ordine di distanza non crescente — estraendoli dalla pila `stack` — e si applica la relazione (2.2). Una scelta implementativa rilevante consiste nel *non* memorizzare esplicitamente le liste dei predecessori $P[w]$: poiché un vicino v di w ne è predecessore se e solo se $d[v] = d[w] - 1$, i predecessori sono

ricalcolati direttamente a partire dal solo vettore `dist`, evitando l'occupazione di memoria e le numerose allocazioni dinamiche che la gestione esplicita di $P[w]$ comporterebbe.

Listing 2.3: Accumulo delle dipendenze (predecessori ricalcolati).

```

1 while (!stack.empty()) {
2     int w = stack.top();
3     stack.pop();
4     for (int v : g.adj[w]) {
5         if (dist[v] == dist[w] - 1)           // v e' predecessore di w
6             delta[v] += (double)sigma[v] / sigma[w] * (1 + delta[w]);
7     }
8     if (w != s)
9         CB[w] += delta[w];
10 }

```

Al termine del ciclo su tutte le sorgenti, trattandosi di un grafo non orientato, ciascuna coppia risulta conteggiata due volte; il valore $C_B[v]$ viene pertanto diviso per 2.

2.7 Ottimizzazioni a costante

L'implementazione adotta alcune ottimizzazioni che, pur lasciando invariata la complessità asintotica $O(nm)$, ne riducono la costante moltiplicativa:

- i buffer `dist`, `sigma` e `delta` sono allocati *una sola volta* all'esterno del ciclo sulle sorgenti e, a ogni iterazione, vengono riscritti tramite `std::fill` anziché riallocati; analogamente la pila `stack` è dichiarata una sola volta e riutilizzata, poiché la fase di accumulo la svuota completamente a fine sorgente. Ciò elimina n coppie di operazioni `malloc/free` per esecuzione;
- l'eliminazione delle liste di predecessori $P[w]$, illustrata in precedenza, rimuove l'allocazione dinamica più onerosa dell'algoritmo;
- per la frontiera della visita in ampiezza si è mantenuto `std::queue` (basato su `std::deque`). È stata valutata l'alternativa di una coda FIFO ad array preallocato con due indici, che eviterebbe le allocazioni a blocchi del `deque`; il confronto sperimentale non ha però evidenziato alcun guadagno (con l'ottimizzazione -O2 il `deque` riusa i blocchi già allocati tra una sorgente e l'altra). Si è perciò preferito `std::queue` per la maggiore leggibilità;
- prima delle misurazioni viene eseguita una breve fase di *warm-up* — alcune esecuzioni preliminari su un grafo di dimensione intermedia, i cui risultati sono scartati — al fine di portare il processore alla frequenza di regime e di precaricare le memorie cache, attenuando l'artefatto di misura noto come “primo punto lento”.

Come mostrato nel Capitolo 3, l'output numerico rimane identico a quello della versione non ottimizzata, a meno del rumore in virgola mobile.

Capitolo 3

Estensione ai grafi pesati: l'algoritmo di Dijkstra

Quanto esposto finora si riferisce a grafi *non pesati*, in cui ogni arco ha costo unitario. In numerose applicazioni reali, tuttavia, agli archi è associato un peso, interpretabile come distanza, tempo o capacità. Il presente capitolo descrive come l'algoritmo di Brandes debba essere modificato per trattare i grafi pesati; la corrispondente implementazione è contenuta nel file `src/brandes_weighted.cpp`.

3.1 Inadeguatezza della BFS

La BFS determina correttamente i cammini minimi soltanto quando tutti gli archi hanno costo identico: in tal caso il primo istante in cui un vertice viene raggiunto corrisponde al suo cammino minimo, e il numero di archi coincide con la distanza. In presenza di pesi questa proprietà decade, poiché un cammino composto da più archi di peso ridotto può risultare complessivamente più breve di un singolo arco di peso elevato. La BFS, che procede per livelli di numero di archi, non individuerrebbe tali cammini.

La struttura generale dell'algoritmo di Brandes — elaborazione di una sorgente per volta, con una fase di calcolo dei cammini minimi seguita dall'accumulo all'indietro delle dipendenze — rimane invariata: a mutare è esclusivamente la prima fase, in cui la BFS è sostituita dall'algoritmo di **Dijkstra**. La relazione ricorsiva sulle dipendenze e la fase di accumulo conservano la medesima logica.

3.2 Le modifiche al codice

Rispetto alla versione non pesata si rendono necessarie tre modifiche.

1. Distanze cumulative e valore di infinito. Il vettore `dist` assume tipo `long long`, dovendo accumulare la somma dei pesi anziché un conteggio di livelli, ed è inizializzato a $+\infty$ in luogo di -1 ; il valore infinito è rappresentato dal massimo valore esprimibile dal tipo.

2. Coda a priorità in luogo della coda FIFO. La `std::queue` è sostituita da una `std::priority_queue` configurata come *min-heap*: in cima si trova costantemente il vertice di distanza provvisoria minima, ossia il successivo da finalizzare. Il modificatore `std::greater` inverte l'ordinamento predefinito — che fornirebbe il massimo — restituendo il minimo.

3. Azzeramento di sigma alla scoperta di cammini più brevi. Quando si individua un cammino *strettamente* più breve verso un vicino, i conteggi di cammini minimi accumulati fino a quel momento si riferiscono a una distanza ormai superata e perdono validità; il valore `sigma[vicino]` viene pertanto azzerato prima di riavviare il conteggio.

Listing 3.1: Fase 1 pesata: Dijkstra con conteggio dei cammini minimi.

```

1 while (!queue.empty()) {
2     auto [d, nodo] = queue.top();
3     queue.pop();
4     if (fatto[nodo]) continue;           // voce obsoleta: scarta
5     fatto[nodo] = 1;
6     stack.push(nodo);
7     for (auto [vicino, w] : g.adj[nodo]) {
8         long long nd = dist[nodo] + w;
9         if (nd < dist[vicino]) {         // cammino piu' corto
10            dist[vicino] = nd;
11            sigma[vicino] = 0;           // i conteggi vecchi non sono
12                piu' minimi
13        }
14        if (nd == dist[vicino])         // altro cammino minimo verso
15            sigma[vicino] += sigma[nodo];
16    }
17 }

```

3.3 La cancellazione pigra

La `std::priority_queue` della libreria standard non mette a disposizione l'operazione di *decrease-key*, ossia l'aggiornamento della priorità di un elemento già inserito. Si ricorre pertanto alla tecnica della *cancellazione pigra* (*lazy deletion*): all'individuazione di una distanza migliore per un vertice si inserisce una nuova coppia (d , vertice), lasciando inalterata la voce precedente. Al momento dell'estrazione, qualora il vertice risulti già finalizzato (in base al flag `fatto`), la voce è obsoleta e viene scartata mediante `continue`. In tal modo ciascun vertice è inserito nella pila `stack` una sola volta, secondo l'ordine di distanza crescente, indispensabile per la successiva fase di accumulo.

La correttezza di questo schema — finalizzare un vertice alla sua *prima* estrazione e scartare le copie successive — è docuta appunto dal min-heap: nel momento in cui un vertice v viene estratto per la prima volta, la sua distanza provvisoria è *già* quella minima definitiva, e nessun cammino più breve potrà essere scoperto in seguito. Si osservi che questa proprietà dipende in modo essenziale dall'ipotesi di **pesi non negativi**: con archi di peso negativo un cammino più breve potrebbe emergere *dopo* la finalizzazione di un vertice, e l'algoritmo di Dijkstra — cancellazione pigra inclusa — fornirebbe risultati errati. È precisamente per questo motivo che Dijkstra richiede pesi non negativi; in presenza di pesi negativi occorre ricorrere ad algoritmi diversi, come quello di Bellman–Ford.

3.4 L'accumulo all'indietro nel caso pesato

La fase di accumulo è strutturalmente identica a quella del caso non pesato: i vertici sono estratti dalla pila `stack` e la dipendenza è distribuita ai predecessori. L'unica differenza risiede nella *condizione di predecessore*: nel caso non pesato un vicino v di w ne è predecessore se

$d_v = d_w - 1$, mentre nel caso pesato la condizione diviene

$$d_v + \text{peso}(v, w) = d_w. \quad (3.1)$$

Analogamente al caso non pesato, i predecessori sono ricalcolati a partire dalle distanze, senza il mantenimento di liste esplicite.

Listing 3.2: Accumulo pesato: cambia solo la condizione di predecessore.

```

1 while (!stack.empty()) {
2     int w = stack.top();
3     stack.pop();
4     for (auto [v, peso] : g.adj[w])
5         if (dist[v] + peso == dist[w]) // v e' predecessore di w
6             delta[v] += (double)sigma[v] / sigma[w] * (1 + delta[w]);
7     if (w != s) CB[w] += delta[w];
8 }

```

3.5 Complessità

Con la coda a priorità l'estrazione del minimo e l'inserimento hanno costo $O(\log n)$ ciascuno e si effettuano $O(m)$ aggiornamenti per sorgente; la singola esecuzione di Dijkstra costa pertanto $O(m \log n)$, a fronte dell' $O(m)$ della BFS. Esteso a tutte le n sorgenti, il costo complessivo risulta

$$O(nm \log n) \quad (3.2)$$

nel caso di heap binario. L'impiego di un *Fibonacci heap* [4], che ammortizza l'operazione di *decrease-key* a $O(1)$, consente di raggiungere il costo teorico $O(nm + n^2 \log n)$ indicato da Brandes. Il fattore logaritmico aggiuntivo rappresenta il costo della maggiore generalità: il caso pesato è intrinsecamente più oneroso di quello non pesato. La Figura 3.1 riporta i tempi misurati per la versione pesata, secondo lo stesso schema sperimentale adottato nel caso non pesato — una curva per ciascuna densità, con sweep su n —; l'andamento conferma il maggiore costo imputabile alla coda a priorità.

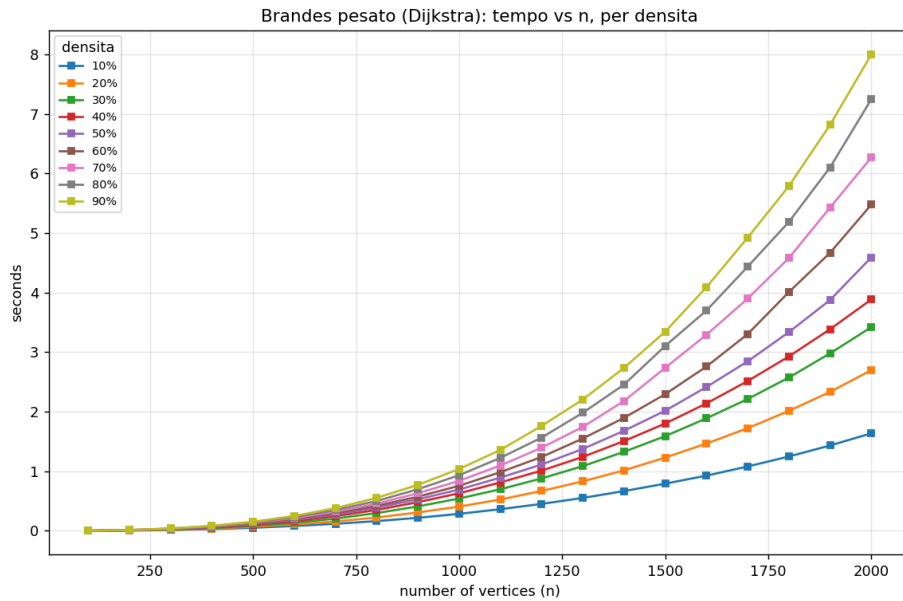


Figura 3.1: Tempo di esecuzione della versione pesata (Dijkstra) in funzione del numero di vertici, una curva per ciascuna densità.

3.6 Verifica ed esempio visivo

Anche la versione pesata è stata validata mediante confronto dei valori di C_B con quelli prodotti da NetworkX, invocata con l'opzione `weight='weight'` oltre a `normalized=False`: i risultati coincidono. La Figura 3.2 riporta un grafo pesato di esempio con i nodi-ponte evidenziati; le etichette sugli archi ne indicano i pesi.

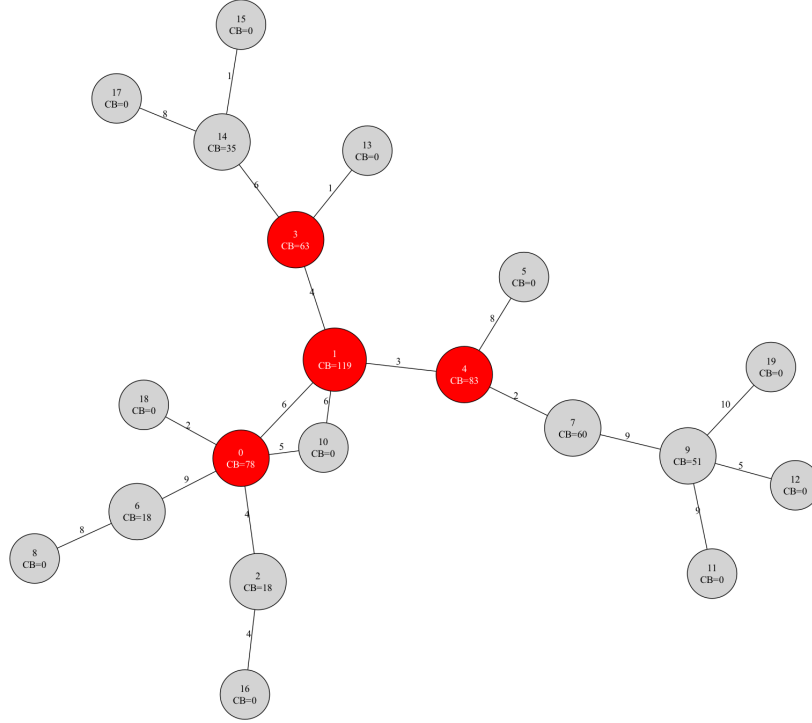


Figura 3.2: Grafo pesato di esempio: in rosso i nodi con betweenness più alta. Le etichette sugli archi indicano i pesi. Rispetto al caso non pesato, i pesi modificano la struttura dei cammini minimi e, di conseguenza, l'insieme dei nodi che vi giacciono.

Capitolo 4

Risultati sperimentali

4.1 Impostazione degli esperimenti

Le prove sono state condotte generando grafi casuali connessi mediante la funzione `genera_grafo` descritta nel Capitolo 2. Al fine di contenere il rumore di misura sono stati adottati i seguenti accorgimenti:

- ciascuna configurazione (n, m) è stata cronometrata ripetendo la misura cinque volte sullo stesso grafo e assumendo il tempo *minimo*: la corsa più veloce è quella meno disturbata dalle interferenze del sistema operativo (scheduling, altri processi), e fornisce dunque la stima più pulita del costo effettivo dell'algoritmo;
- i tempi sono stati rilevati tramite `std::chrono::high_resolution_clock`, cronometrando la sola invocazione di `brandes`;
- è stata anteposta una fase di *warm-up* per stabilizzare la frequenza del processore.

I risultati sono salvati in un file `risultati.csv` ed elaborati da uno script Python (`plot.py`) per la produzione dei grafici; la variante pesata segue il medesimo protocollo, con output su `risultati_w.csv`.

4.2 Verifica di correttezza

Prima di procedere all'analisi dei tempi è indispensabile accertare la correttezza dei valori calcolati dall'implementazione. I valori di C_B prodotti dal codice C++ sono stati confrontati con quelli forniti dalla funzione `betweenness centrality` della libreria NetworkX [5] (con `normalized=False`, così da adottare la medesima scala) su un grafo di prova con $n = 15$ e $m = 20$.

Tabella 4.1: Confronto C_B : implementazione C++ vs NetworkX (estratto).

nodo	C++	NetworkX	differenza
1	36.3333	36.3333	3.3×10^{-11}
3	36.3333	36.3333	3.3×10^{-11}
0	33.6667	33.6667	3.3×10^{-11}
5	16.3333	16.3333	3.3×10^{-11}
11	0.0000	0.0000	0.0
13	0.3333	0.3333	3.3×10^{-13}

La differenza massima riscontrata sull'insieme dei nodi è dell'ordine di 10^{-11} , riconducibile al solo rumore in virgola mobile: le due implementazioni producono il medesimo risultato. La verifica è stata ripetuta sulla versione ottimizzata del codice (buffer riutilizzati, predecessori ricalcolati), con esito parimenti coincidente, a conferma del fatto che le ottimizzazioni a costante non ne alterano la correttezza.

4.3 Interpretazione visiva della betweenness

Allo scopo di attribuire un significato intuitivo alla misura, la Figura 4.1 mostra un grafo di esempio in cui i nodi con betweenness più elevata — i cosiddetti *ponti* — sono evidenziati in rosso.

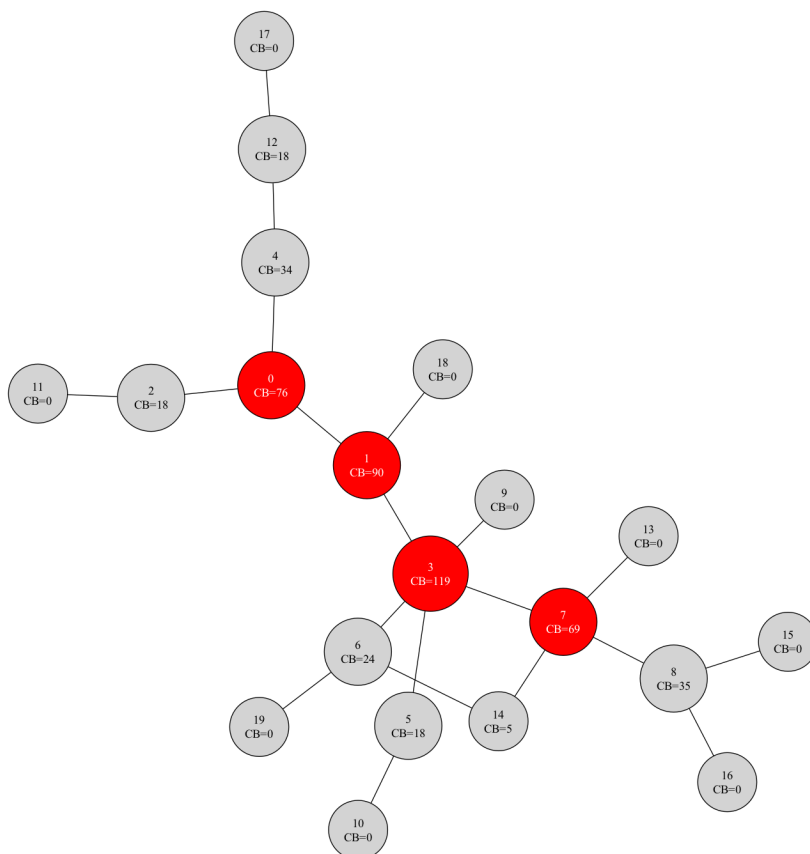


Figura 4.1: Grafo di esempio: in rosso i nodi con betweenness più alta. Si nota come i ponti non siano necessariamente i nodi con più connessioni, ma quelli che giacciono sui passaggi obbligati tra le diverse zone del grafo. I nodi foglia, come il nodo 11, hanno betweenness nulla.

4.4 Analisi della complessità

Per verificare la complessità $O(nm)$ è stato misurato il tempo di esecuzione al variare di n e m . Riportando il tempo in funzione del prodotto $n \cdot m$ si ottiene un andamento sostanzialmente lineare, con coefficiente di determinazione $R^2 \approx 0.93$ una volta esclusi i grafi disconnessi. Una rappresentazione complementare, con il tempo in funzione di n per densità fissata, è riportata nella Figura 4.2 della sezione successiva.

4.4.1 Il sovraccarico lineare per sorgente

Un'osservazione rilevante emersa dagli esperimenti riguarda il comportamento a m fissato: mantenendo costante il numero di archi e incrementando n , il tempo non cresce in modo lineare bensì quadratico. La causa risiede nel fatto che il costo effettivo per ciascuna sorgente non è $O(m)$ ma $O(n + m)$, poiché l'inizializzazione dei buffer (`dist`, `sigma`, `delta`) richiede $O(n)$ operazioni a ogni sorgente. Il costo complessivo è dunque

$$O(n \cdot (n + m)) = O(n^2 + nm), \quad (4.1)$$

e quando m è piccolo rispetto a n il termine n^2 risulta dominante. Ciò giustifica la curvatura osservata e mette in evidenza come la stima $O(nm)$ presupponga implicitamente grafi sufficientemente densi ($m \gtrsim n$).

4.5 Riproduzione dell'esperimento di Brandes

Infine, è stato riprodotto lo schema sperimentale dell'articolo originale: uno sweep del numero di vertici da 100 a 2000, con una curva per ciascuna densità compresa tra il 10% e il 90%, secondo lo stesso impianto della Figura 3 di Brandes [1]. Il risultato è riportato nella Figura 4.2.

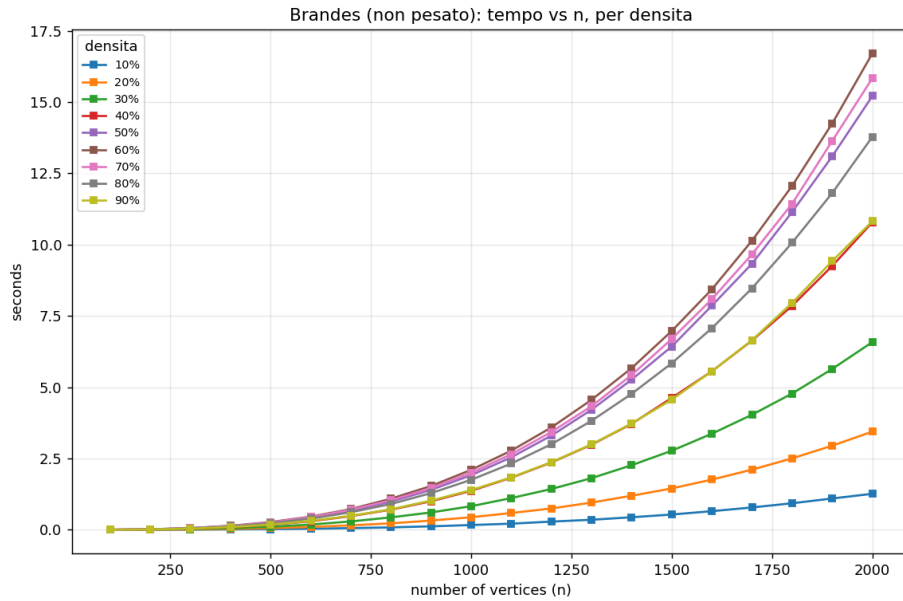


Figura 4.2: Tempo di esecuzione (caso non pesato) in funzione del numero di vertici, una curva per ciascuna densità dal 10% al 90%: riproduzione della Figura 3 dell'articolo originale di Brandes.

Capitolo 5

Conclusioni

5.1 Sintesi del lavoro

In questa tesina è stato implementato in C++ l'algoritmo di Brandes per il calcolo della betweenness centrality su grafi non orientati, nelle due varianti *non pesata* (BFS) e *pesata* (Dijkstra). Il lavoro ha riguardato quattro aspetti complementari:

- **correttezza:** i valori prodotti coincidono con quelli della libreria di riferimento NetworkX a meno del rumore in virgola mobile ($\sim 10^{-11}$), sia nella versione base sia in quella ottimizzata, e anche nella variante pesata (confronto con `weight='weight'`);
- **efficienza:** l'analisi sperimentale conferma la complessità $O(nm)$ prevista dalla teoria per il caso non pesato, evidenziando inoltre il sovraccarico lineare per sorgente che conduce al costo complessivo $O(n^2 + nm)$;
- **estensione ai grafi pesati:** la sostituzione della visita in ampiezza con l'algoritmo di Dijkstra e una coda a priorità consente di trattare archi con peso arbitrario, a fronte di un fattore logaritmico aggiuntivo ($O(nm \log n)$ con heap binario), come confermato dai tempi misurati al variare della densità;
- **ottimizzazione:** il riutilizzo dei buffer di memoria e l'eliminazione delle liste di predecessori riducono la costante moltiplicativa senza modificare la complessità asintotica né i risultati.

In conclusione, l'algoritmo di Brandes si conferma una soluzione elegante ed efficiente al problema della betweenness centrality: il passaggio da uno spazio $O(n^2)$ a uno spazio lineare $O(n+m)$, ottenuto grazie all'accumulo ricorsivo delle dipendenze, è ciò che ne rende possibile l'uso su reti di grande dimensione.

Bibliografia

- [1] U. Brandes, *A Faster Algorithm for Betweenness Centrality*, Journal of Mathematical Sociology, 25(2):163–177, 2001.
- [2] L. C. Freeman, *A Set of Measures of Centrality Based on Betweenness*, Sociometry, 40(1):35–41, 1977.
- [3] J. M. Anthonisse, *The Rush in a Directed Graph*, Stichting Mathematisch Centrum, Amsterdam, 1971.
- [4] M. L. Fredman, R. E. Tarjan, *Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms*, Journal of the ACM, 34(3):596–615, 1987.
- [5] A. Hagberg, D. Schult, P. Swart, *Exploring Network Structure, Dynamics, and Function using NetworkX*, Proceedings of the 7th Python in Science Conference, 2008.